Engenharia de Caracteristicas para Aprendizagem Computacional / Engenharia de Atributos

Models and their evaluation

Licenciatura em Enheraria Informática / Licenciatura em Inteligência Artificial e Ciência de Dados / Licenciatura em Engenharia e Ciência de Dados
Edition 2025-26

Marco Simões

ML Process

Model Learning

- **Goals**
  - Tackle the respective learning problem by **creating a good model from data** according to the defined requirements and learning problem
    - **Requirements**
      - **Accuracy**
      - **Interpretability**
      - **…**
    - **Learning problem**
      - Classification, regression, association, clustering

ML Process

Model Learning

- **Learning Problems**
  - **Classification**
    - E.g., decision-tree
  - **Regression**
    - E.g., linear regression
  - **Association**
    - E.g., Apriori
  - **Clustering**
    - E.g., K-means clustering
  - …

ML Process

Model Learning

- **How?**
  - Define the **training** and **test sets**
    - Train set: used to learn the model
    - Test set: used to evaluate the model on unseen data

    - See section on Model Evaluation

ML Process

Model Learning

- **How?**
  - **Select and compare different models**
    - Performance comparison (see Model Evaluation)
      - *Naïve Bayes* is often used as baseline algorithm; *C4.5* or *SVMs*, for example, often perform better (see chapter Algorithms)
      - Interpretability comparison

It **has to be shown** empirically from realistic examples that a **particular learning technique is necessarily better than the others**.

When faced with ***N* equivalent techniques**, Occam's razor advises to use the **simplest of them**.

ML Process

Model Learning

- **How?**
  - Perform model **parameter tuning**
    - *Number of neighbors* in *k*-Nearest Neighbors
    - *Kernel type*, *complexity*, *epsilon*, *gamma* in SVMs
    - *Confidence factor* in C4.5
    - …

ML Process

Model Learning

- Important Question
  - What is the effect of the **number of training examples**, **features**, **number of model parameters**, etc., in the learning performance?
    - Not many definitive answers…
    - Too many parameters relative to the number of observations → overfitting might happen
    - Too many features → curse of dimensionality
      - Convergence of any estimator to the true value is very slow in a high-dimensional space

ML Process

Model Evaluation

- **Goals**
  - Evaluate **model generalization capability** in a **systematic way**
    - **How the model will perform** on **unseen, <u>realistic</u>, data**,
      - E.g., sometimes test sets are “carefully” chosen (and not in a good way ☺)
  - Evaluate **how** one model **compares** to **another**
  - Show that the **learning method** leads to **better performance** than the one achieved **without learning**
    - E.g., making credit decisions using a decision tree might lead to better results (70%) than the one achieved by humans (50%)

ML Process

Model Evaluation

- **How?**
  - Check which **kinds of errors** are more prevalent and why
    - What classes/variables show low accuracy? Why?
      - important features might be missing
    - Where is the root of the problem: classifier/regressor, extracted features, method employed for extraction, feature selection approach?

ML Process

Model Evaluation

- **How?**
  - Use a **separate test set**
    - Predict the behavior of the model in unseen data
  - Use an adequate **evaluation strategy**
    - E.g., stratified 10-fold cross-validation
  - Use an adequate **evaluation metric**
    - Regression: **RMSE** and **R^2** are typically used
    - Classification: **Precision, recall and F-measure** are standard

ML Process

Model Evaluation

- **Common Requirements**
  - **Accuracy**
  - **Interpretability**
  - There is often a **trade-off** between accuracy and interpretability
    - E.g., decision tree: trade-off between succinctness (smaller trees) versus classification accuracy
    - E.g., rule induction algorithms might lead to weaker results than an SVM

ML Process

Model Evaluation

- Basic Definitions
  - **Training set**
    - Set of examples used to **learn the model**, i.e., to train the classifier, regressor, etc.
  - **Test set**
    - Independent, **unseen**, **examples** used to **evaluate the learnt model**

ML Process

Model Evaluation

The **test set** must **not be used in any way** to create the model!

Beware of feature normalization, feature selection, parameter optimization, etc.

The **larger the training set**, the **better the classifier**!

Diminishing returns after a certain volume is exceeded.

The **larger the test set**, the **better the accuracy estimate**!

ML Process

Model Evaluation

- Why test on unseen data?
  - Example: learn an unknown linear function $y = 3x + 4$, with some measurement error
    - You don't know the underlying concept, so you acquire some measurements
      - x in the [0, 10] interval, and the corresponding y values
    - Then, you blindly experiment with 2 models
      - One linear model
      - Another 10th order polynomial model

ML Process

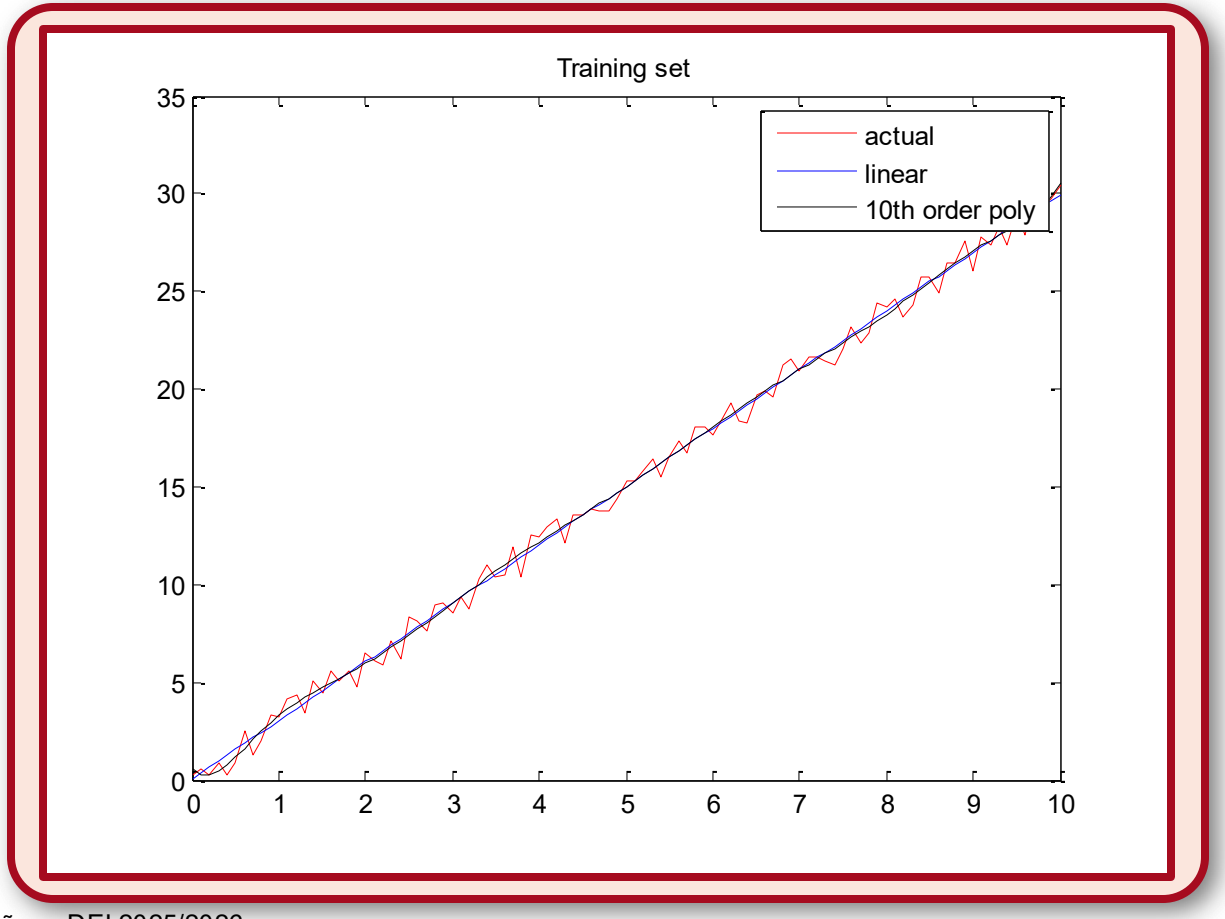
Model Evaluation

- Why test on unseen data?
  - Example
    - You measure mean squared error (MSE) and get 0.24 for the linear model and 0.22 for the 10th order polynomial model
    - You naturally conclude that the **polynomial model performs slightly better**

ML Process

Model Evaluation

- Why test on unseen data?



ML Process

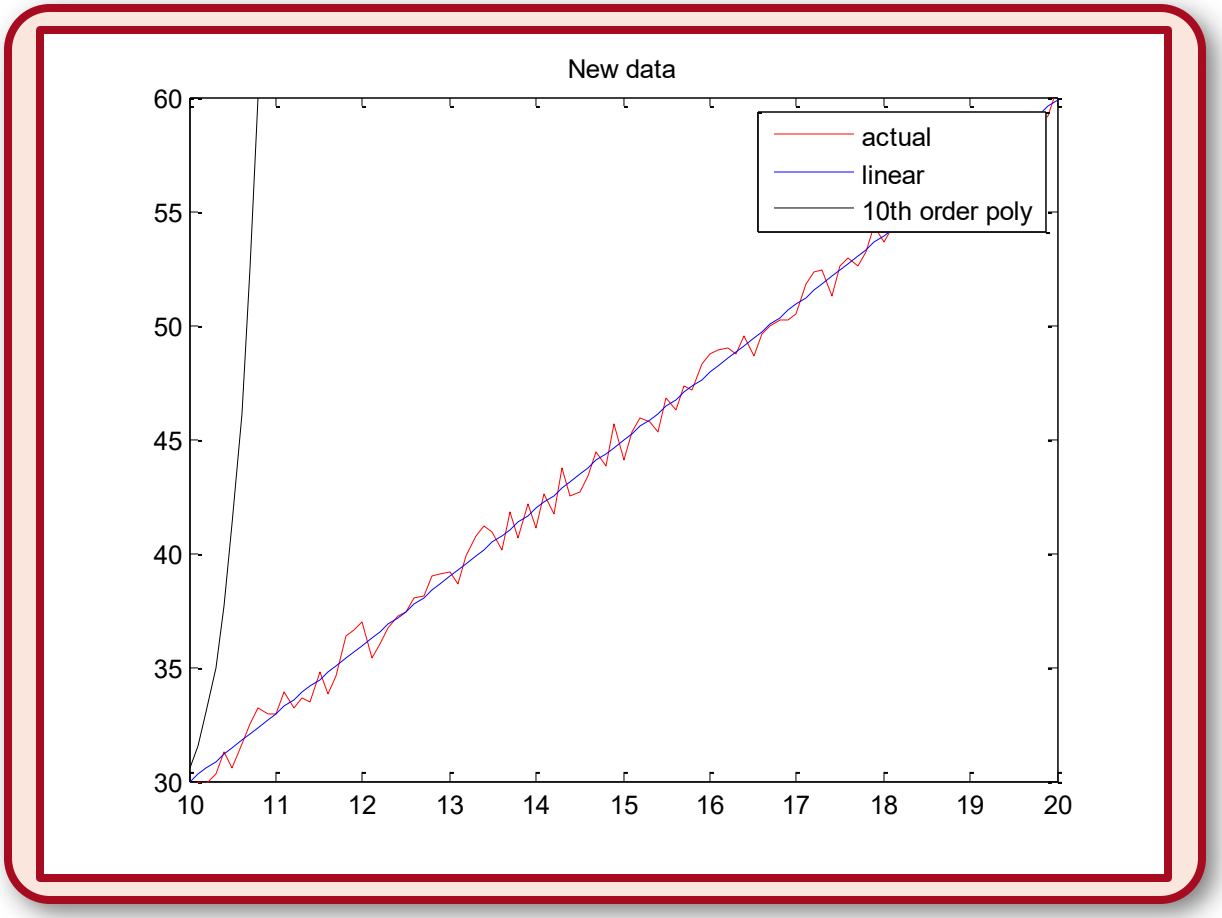
Model Evaluation

- Why test on unseen data?
  - Example
    - You then repeat data acquisition using a different range, e.g., *x* in [10, 20], and compare the observed *y* with the outputs from the two models
    - To your surprise, you observe a 0.19 MSE for the linear model and a 1.07E+12 MSE for the polynomial model!!!
    - That's **overfitting**: your polynomial model was overly adjusted to the training data

ML Process

Model Evaluation

- Why test on unseen data?



ML Process

Model Evaluation

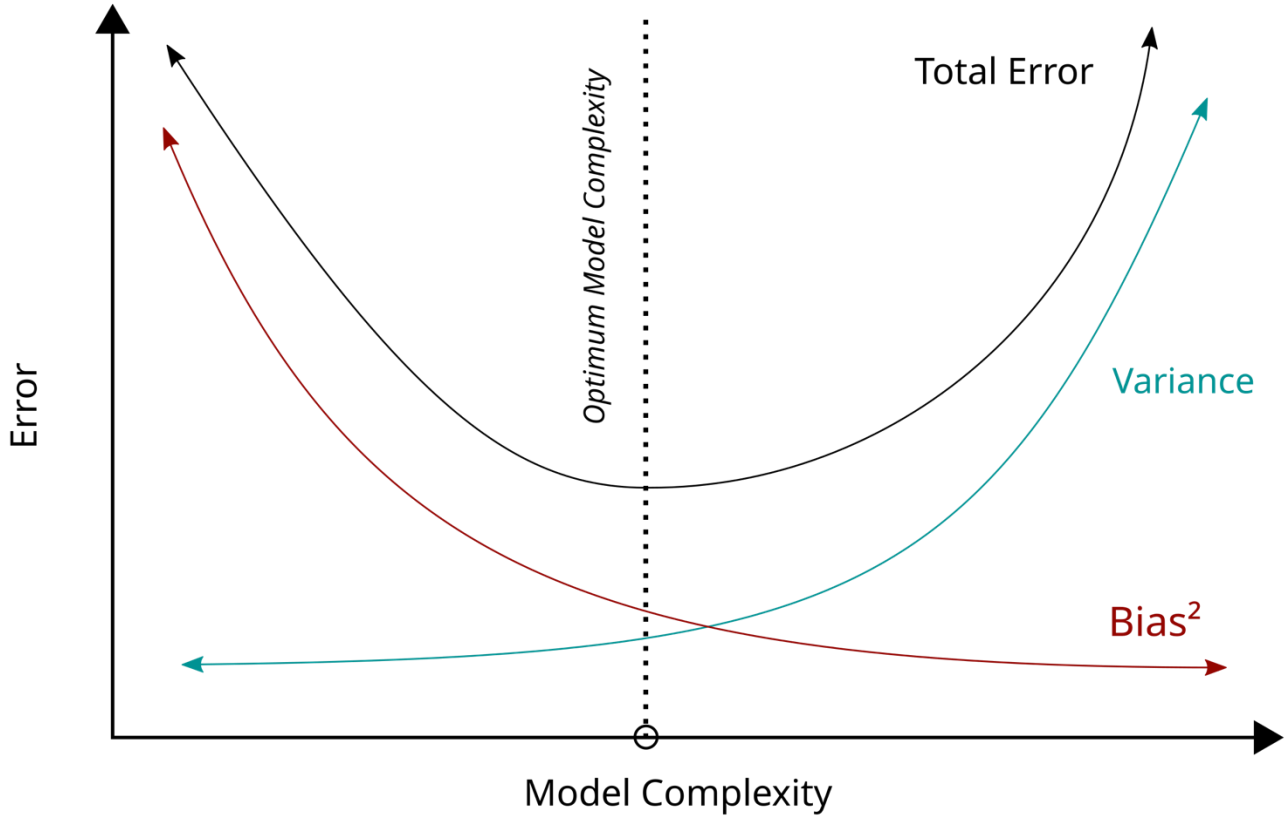
- Why test on unseen data?
  - **Answer: minimize overfitting**
    - **Overfitting** occurs when the **model** is **overly adjusted to the data** employed in its creation, and so the model **"learns beyond the concept",** e.g., learns noise or some other concept, but not the underlying concept
  - **Answer:** have a **realistic estimate** of model **performance on unseen data**

A **more accurate representation in the training set** is **not necessarily** a **more accurate representation of the underlying concept!**

ML Process

Model Evaluation

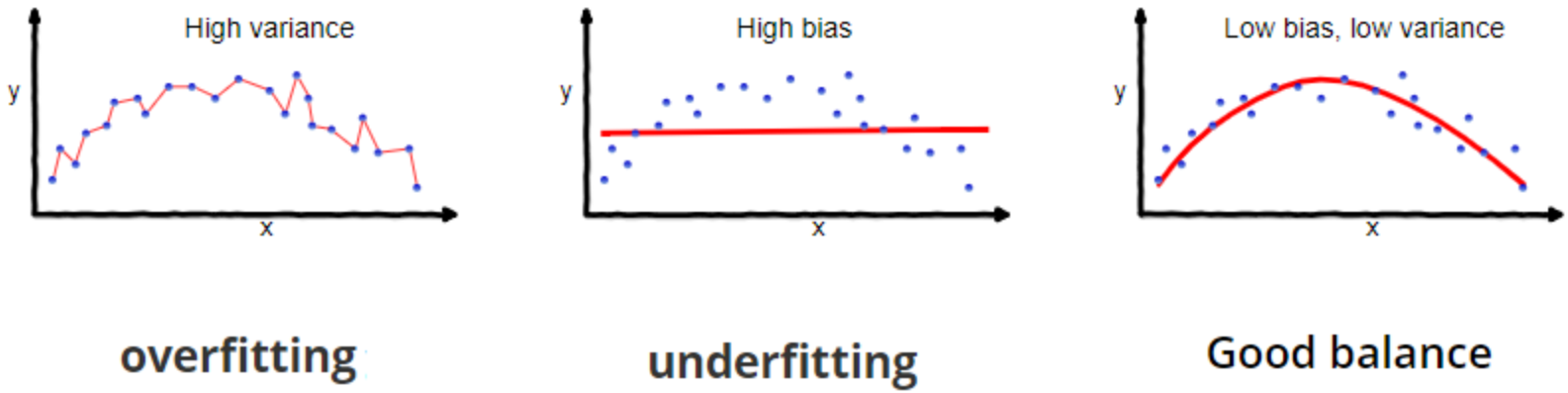
- Why test on unseen data?



ML Process

Model Evaluation

- Why test on unseen data?



https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

ML Process *Model Evaluation*

- Basic Definitions
  - **Bias-Variance Dilemma**
    - **Bias**
      - **Difference** between this estimator's **expected value** and the **true value**
        - » E.g., we are trying to estimate model performance using a limited dataset and we want the estimated performance to be as close as possible to the real performance
        - » **Unbiased** estimator: zero bias
      - **Sources of bias**
        - » **Strong model assumptions**, e.g., assume a linear model (although input-output relationships might be non-linear
          - » → High modelling error → **underfitting**
        - » **Training set** with **lack of diversity**, e.g., most samples from one class, lack of diversity in samples within the same class

ML Process

Model Evaluation

- Basic Definitions
  - **Bias-Variance Dilemma**
    - **Variance**
      - **Variability of estimator**: how much the estimate of the target function will change if different training data is used
        - » High variance: sensitivity to small changes in the training set
      - **Non-linear algorithms** → have a lot of flexibility → **high variance**
      - Typically associated with overfitting
    - **Dilemma** or **trade-off**
      - In practice, there is often a trade-off between minimizing bias and variance simultaneously

ML Process

Model Evaluation

- E**valuation** S**trategies** (see [Refaeilzadeh et al. 2009])
  - **Training set** (a.k.a **Resubstitution Validation**)
    - Idea
      - Evaluate model performance using some metric (see section Evaluation Metrics) resorting **only to the training set**
        - » I.e., train using the entire dataset, evaluate using the same entire dataset (i.e., validate with resubstitution)

ML Process *Model Evaluation*

- **Evaluation Strategies**
  - **Training set** (a.k.a **Resubstitution Validation**)
    - Limitations
      - **Accuracy** on the training set **overly optimistic** → not a good indicator of the performance on the test set
      - **Overfitting** often turns out
      - → **High bias**, **high variance**
    - Advantages
      - Have an idea of **data quality**
        - » Low training accuracy in the training set may indicate poor data quality, missing relevant features, dirty data, etc.

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **Hold-out**
    - Idea
      - Separate the entire set into two non-overlapping sets
        - » **Training set**: typically, **2/3** or **70-30**
          - » Rule of thumb: training set should be more than 50%
        - » **Test set**: unseen data, typically **1/3** (data *held out* for testing purposes and not used at all during training)
      - **Learn** (train) the model in the **first set**
      - **Evaluate** in the **second set**

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **Hold-out**
    - Limitations
      - Results **highly dependent** on the **train/test split**
        - » Training/test sets might **not be representative**
          - » In the limit, the training set might contain no samples of a given class…
          - » The cases in the **test set** might be **too easy** or **too hard** to classify
      - Again, **high bias** and **high variance**

- **Evaluation Strategies**
  - **Hold-out**
    - Limitations
      - Requires a **large dataset**, so that the **test set** is **sufficiently representative**
        - » Often unavailable…
          - » Manual annotation typically requires specialized human expertise and takes time → datasets are often small

- **Evaluation Strategies**
  - **Stratified Hold-out**
    - Idea
      - In classification, **hold-out** with **classes balanced** across the training and test set (i.e., **stratification**)
        - » Promotes sample **representativity** in the two sets, due to class balancing
    - Advantages
      - May **reduce bias**, due to higher representativity, but it is not as low as could be, e.
    - Limitations
      - **High variance** still unsolved
      - Still requires a **large dataset**

- **Evaluation Strategies**
  - **Repeated (Stratified) Hold-out**
    - Idea
      - **Repeat** the training and testing process **several times** with **different random samples** and **average the results**
        - » Typically, between 10 and 20 repetitions
    - Advantages
      - **Lower variance** observed in performance estimation due to repetition

- **Evaluation Strategies**
  - **Repeated (Stratified) Hold-out**
    - Limitations
      - **Bias could be lower**
        - » Typically, **some data may be included in the test set multiple times** while **others are not included at all**
        - » Some data may **always fall in the test set** and **never contribute to the learning phase**

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **(Repeated) (Stratified) Train-Validate-Test (TVT)**
    - Idea
      - 3 independent datasets
      - **Training set**: learn the model
      - **Validation set**: used to evaluate parameter optimization, feature selection, compare models, etc.
      - **Test set**: used to evaluate the final accuracy of the optimized model
      - Typical split percentages: **40-30-30** (train-validate-test sets, respectively)

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **(Repeated) (Stratified) Train-Validate-Test (TVT)**
    - Limitations
      - Again, requires an even **larger dataset**, so that **test** and **validation sets** are **sufficiently representative**
      - Same **bias** and **variance limitations**

ML Process *Model Evaluation*

- **Evaluation Strategies**
  - **Repeated Stratified K-fold Cross-Validation**
    - Idea
      1. **Randomly separate** the entire set into a number of stratified ***k* equal-size folds** (partitions)
      2. **Train** using ***k*-1folds**, **test** using the **remaining fold**
      3. **Repeat training and testing** (step 2) *k* times, alternating each fold in the test set
      4. **Repeat steps 1 to 3 a number of times** (reshuffle and restratify data)
         » **Typically, 10 to 20** times
      5. **Average** the **results**

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **Repeated Stratified K-fold Cross-Validation**

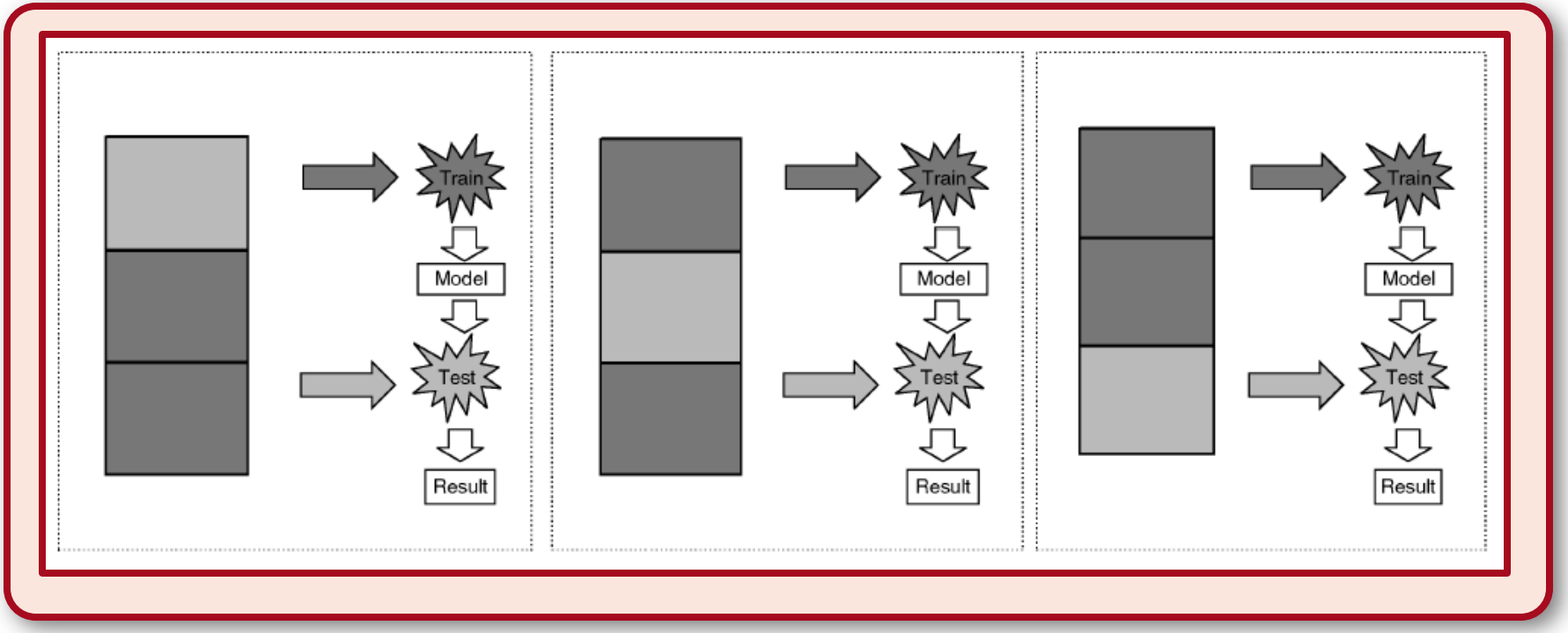


Illustration of 3-fold cross validation [Refaeilzadeh et al. 2009]

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **Repeated Stratified <u>K-fold Cross-Validation</u>**
    - Recommended *k*
      - High *k*: lower bias, higher variance
      - Low *k*: higher bias, lower variance
      - Typically, ***k* = 10**, i.e., **10-fold cross validation**
        - » Mostly, **empirical result**
        - » Good **bias-variance trade-off**

Method of choice in most practical situations!

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **Repeated Stratified K-fold Cross-Validation**
    - Advantages
      - Guarantees that **all samples** appear in the **test set** → **lower bias**
        - » Average repeated, stratified, 10-fold cross-validated performance considered a **good estimate of model performance** on **unseen data**
      - **Lower variance**
        - » Repetition of the experiment shows low performance variance
      - **Good bias-variance trade-off**
      - Useful for performance prediction based on **limited data**

- **Evaluation Strategies**
  - **Repeated Stratified K-fold Cross-Validation**
    - Limitations
      - **Computational cost**
        - » 10 x 10-folds is expensive for large and complex datasets, or complex model learning algorithms
          - » → 5-fold might be used in such cases

ML Process *Model Evaluation*

- **Evaluation Strategies**
  - **Nested RS K-fold Cross-Validation**
    - Idea
      - Nest repeated (R) stratified (S) K-fold cross-validation (CV) if **model structure** or **parameter tuning** is a goal
        - » E.g., tune SVM parameters, perform feature selection, find out how many hidden layer neurons in an MLP
    - **Test set should never be used during training!**
      - → use an **outer CV** for **testing** and an **inner CV** for **structure/parameter learning**
        - » Conceptually similar to the **train-validate-test** strategy, but without bias and variance limitations

ML Process *Model Evaluation*

- **Evaluation Strategies**
  - **Leave-One-Out Cross-validation**
    - Idea
      - Cross- validation (CV) where each **fold contains only one sample**, i.e., *n*-fold CV (*n* = number of samples)
    - Advantages
      - **Deterministic**: no random selection involved → no need for repetition
      - **Low bias**
      - **Greatest** possible **amount** of data for **training** → increases the chance that the classifier is a good one
      - **Adequate** for particularly **small datasets**

- **Evaluation Strategies**
  - **Leave-One-Out Cross-validation**
    - Limitations
      - **High variance**
      - **Non-stratified test set**
        - » A model based on the majority class will always make a wrong prediction in a 2-class problem…
      - **Computational time**
        - » Might be infeasible in very large datasets

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **(Repeated) Bootstrap**
    - Idea
      - Based on the procedure of **sampling <u>with</u> replacement**, i.e., select the same sample more than once
      - **Training set**: get the original dataset and **sample with replacement**
        - » This set will typically contain 63.2% of all samples (so, the method is often termed **0.632 bootstrap**
      - **Test set**: **unselected samples** (36.8%)
      - Error estimation: use the two sets
        - » $e = 0.632 \times e_{\text{test instances}} + 0.368 \times e_{\text{training instances}}$

ML Process

Model Evaluation

- **Evaluation Strategies**
  - **(Repeated) Bootstrap**
    - Advantages
      - Probably the **best way** for accuracy estimation in **very small datasets**
  - **Expert examination**
    - Might be necessary, e.g., in **unsupervised learning**

ML Process *Model Evaluation*

- Performance M**etrics**
  - **Classification**
    - Precision/Recall, F-measure, error rate
  - **Regression**
    - Root mean squared error, correlation, R2 statistics
  - **Clustering**
    - If labels are available, compare created clusters with class labels; otherwise, expert evaluation
  - **Association**
    - Expert evaluation

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**

Sample nr.	Real class	Predicted class
1	M	M
2	M	M
3	M	B
4	M	M
5	M	M
6	M	R
7	M	M
8	B	B
9	B	B
10	B	R
11	B	B
12	B	B
13	R	R
14	R	R
15	R	M
16	R	B
17	R	R

Example:

Animal classification: mammal (M), bird (B) or reptile (R)

Test set:

Mammal: 7 samples (2 errors)
Bird: 5 samples (1 error)
Reptile: 5 samples (2 errors)

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Confusion matrix** (or **contingency table**)
      - Matrix distribution of classifications through classes
        - » **Lines**: distribution of real **samples**
        - » **Columns**: distribution of **predictions**
      - Goal: **zeros outside the diagonal**
      - Example: animals

Actual \ Predicted as	M	B	R
M	5	1	1
B	0	4	1
R	1	1	3

Actual \ Predicted as	M	B	R
M	71.43%	14.23%	14.23%
B	0%	80%	20%
R	20%	20%	60%

ML Process *Model Evaluation*

- Performance M**etrics**
  - **Classification problems**
    - **Confusion matrix** (CM) or **contingency table**
      - Some properties
        - » **Sum across line**: number of **actual samples** of the class in that line
          - » E.g., line M: 5 + 1 + 1 = 7 actual mammals
        - » **Sum across column**: number of **predictions** that fall in that class
          - » E.g., column M: 5 + 0 + 1 = 6 samples predicted as mammals
      - Confusion matrix more useful when **percentages** shown
        - » E.g., line M: 5/7, 1/7, 1/7 = 71.43%, 14.23%, 14.23%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Error rate**
      - Proportion of errors made over the test set

$$error\ rate = \frac{\#wrong\ classifications}{N}$$

        » # = "number of"
        » *N*: total number of test samples
      - Goal: **minimize error rate**
      - Example: animals
        » 5 errors in 17 cases → 5/17 = 29.4% error rate
          » # errors = **sum of non-diagonal** values in the CM

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Accuracy** (or **success rate**)
      - Proportion of **correct classifications** over the test set

$$accuracy = 1 - error\ rate = \frac{\#correct\ classifications}{N}$$

      - Goal: **maximize accuracy**
      - Example: animals
        - » 12 correct classifications in 17 cases → 12/17 = 70.59% accuracy
          - » # correct = **sum of diagonal** values in the CM

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class True Positive (TP) rate**
      - **True positives**: samples predicted to belong to class *i* that actually belong to that class, i.e., **class accuracy**

$$TP\ rate_i = \frac{TP_i}{N_i}$$

        » TP_i: number of true positives of class *I*
        » N_i: number of samples that belong to class *i*
      - Goal: **maximize TP rate**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class True Positive (TP) rate**
      - TP_i: diagonal value of the confusion matrix
      - N_i: sum of line i

		Predicted as		
		M	**B**	**R**
Actual	**M**	TP_M		
	B		TP_B	
	R			TP_R

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class True Positive (TP) rate**
      - Example: animals
        - » Class M: $TP_M = 5$; $N_M = 7 \rightarrow TP\ rate_M = 71.43\%$
        - » Class B: $TP_B = 4$; $N_B = 5 \rightarrow TP\ rate_B = 80\%$
        - » Class R: $TP_R = 3$; $N_R = 5 \rightarrow TP\ rate_R = 60\%$

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Global True Positive (TP) rate**
      - **Weighted average** of **TP rates** for individual classes

$$TP\ rate = \frac{1}{N}\sum_{i=1}^{C} N_i \cdot TP\ rate_i$$

        » *C*: number of classes
      - The same as **accuracy**
      - Animals example
        » *TP rate* = 1/17 x (7 x 71.43 + 5 x 80 + 5 x 60) = 70.59%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class False Positive (FP) rate**
      - **False positives**: samples that do not belong to class *i* but that are predicted as that class

$$FP\ rate_i = \frac{FP_i}{\overline{N_i}}$$

» FP_i: number of false positives of class *I*

» $\overline{N_i}$: number of samples that do not belong to class *I*

- Goal: **minimize FP rate**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class False Positive (FP) rate**
      - FP_i: sum of column *i* cells, except for the diagonal
      - $\overline{N_i}$: sum of all lines, except for *i*

		Predicted as		
		M	**B**	**R**
Actual	**M**	TP_M	$\boldsymbol{FP_B}$	$\boldsymbol{FP_R}$
	B	$\boldsymbol{FP_M}$	TP_B	$\boldsymbol{FP_R}$
	R	$\boldsymbol{FP_M}$	$\boldsymbol{FP_B}$	TP_R

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class False Positive (FP) rate**
      - Animals example
        - » Class M: FP_M = 1 (sample 15); $\overline{N_M}$ = 5 + 5 = 10 (5 B + 5 R)
          → $FP\ rate_M$ = 10%
        - » Class B: FP_B = 2; $\overline{N_B}$ = 7 + 5 = 12 → $FP\ rate_B$ = 16.67%
        - » Class R: FP_R = 2; $\overline{N_R}$ = 7 + 5 =12 → $FP\ rate_R$ = 16.67%

- Performance M**etrics**
  - **Classification problems**
    - **Global False Positive (FP) rate**
      - **Weighted average** of **FP rates** for individual classes

$$FP\ rate = \frac{1}{\sum_{i=1}^{C} \overline{N_i}} \sum_{i=1}^{C} \overline{N_i} \cdot FP\ rate_i$$

      - Animals example
        » *FP rate* = 1/34 x (10 x 10 + 12 x 16.67 + 12 x 16.67) = 14.71%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class Precision**
      - **Fraction** of **samples predicted as class *i*** that indeed **belong to class *i***
      - Related to the **incidence of false alarms**

$$precision_i = \frac{TP_i}{TP_i + FP_i}$$

      - Denominator: **sum of column *i***

Attention! In this context, **precision** is very **different** than **accuracy**!!!

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class Precision**
      - Animals example
        - » Class M: $precision_M$ = 5 / (5 + 1) = 83.33%
        - » Class B: $precision_B$ = 4 / (4 + 2) = 66.67%
        - » Class R: $precision_R$ = 3 / (3 + 2) = 60%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Global Precision**
      - **Weighted average** of **precision** for individual classes

$$precision = \frac{1}{N}\sum_{i=1}^{C} N_i \cdot precision_i$$

      - Animals example
        - » *precision* = 1/17 x (7 x 83.33 + 5 x 66.67 + 5 x 60) = 71.57%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class Recall**
      - **Fraction** of **samples of class *i*** that are **correctly classified**
        - » i.e., **accuracy of class** *i*, the same as TP rate$_i$

$$recall_i = \frac{TP_i}{TP_i + FN_i}$$

        - » FN_i: number of **false negatives** of class *I*
          - » Elements of class I that are incorrectly classified, i.e., *falsely* classified as *not* belong to that class (hence, false negatives)
      - Denominator: **sum of line *i***

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class Recall**
      - Animals example
        - » Class M: $recall_M$ = 5 / (5 + 2) = 71.43%
        - » Class B: $precision_B$ = 4 / (4 + 1) = 80%
        - » Class R: $precision_R$ = 3 / (3 + 2) = 60%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Global Recall**
      - **Weighted average** of **recall** for individual classes
        - » **= accuracy = TP rate**

$$recall = \frac{1}{N}\sum_{i=1}^{C} N_i \cdot recall_i$$

      - Animals example
        - » *recall* = 1/17 x (7 x 71.43 + 5 x 80 + 5 x 60) = 70.59%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class Precision vs Recall**

CM focused on columns (precision)

Actual \ Predicted as	M	B	R
M	TP_M	FP_B	FP_R
B	FP_M	TP_B	FP_R
R	FP_M	FP_B	TP_R

CM focused on lines (recall)

Actual \ Predicted as	M	B	R
M	TP_M	FN_M	FN_M
B	FN_B	TP_B	FN_B
R	FN_R	FN_R	TP_R

- Performance M**etrics**
  - **Classification problems**
    - **Class F-measure**
      - **F-measure**: a.k.a. F-score or F1-score or balanced F1-score
      - **Combination** of **precision** (related to the incidence of false alarms) and **recall** (class accuracy) into a **single metric**
        - » **Harmonic mean** of precision (P) and recall (R)

$$F - measure_i = 2\frac{P_i \cdot R_i}{P_i + R_i}$$

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Class F-measure**
      - Animals example
        - » Class M: $F\text{-}measure_M$ = 76.92%
        - » Class B: $F\text{-}measure_B$ = 72.72%
        - » Class R: $F\text{-}measure_R$ = = 60%

- Performance M**etrics**
  - **Classification problems**
    - **Global F-measure**
      - **Weighted average** of **F-measure** for individual classes

$$F\text{-}measure = \frac{1}{N}\sum_{i=1}^{C} N_i \cdot F\text{-}measure_i$$

      - Animals example
        - » *F-measure* = 1/17 x (7 x 76.92 + 5 x 72.72 + 5 x 60) = 70.71%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Most common metrics**
      - Precision, recall, F-measure
      - **Sensitivity** and **specificity** (often used in medicine)
        - » **Sensitivity** = recall = TP rate
        - » **Specificity** = 1 – FP rate
          - » Proportion of people with a disease with a negative test
    - **Other metrics**

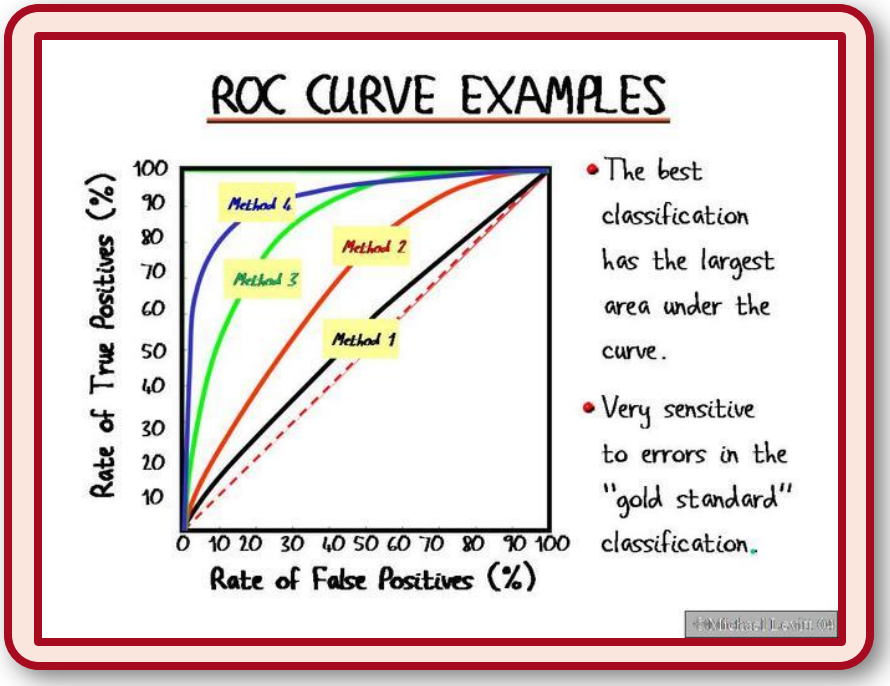
ML Process *Model Evaluation*

- Performance M**etrics**
  - **Classification problems**
    - **Other metrics** [Witten et al., 2011, pp. 166-178]
      - **Cost-Benefit Analysis**
      - **Lift Charts**
      - **ROC** (Receiver Operating Characteristic) **curves**
      - **AUC**: **Area U**nder the ROC **C**urve
      - **Precision-recall curves**
      - **Cost curves**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems:** other metrics
    - https://en.wikipedia.org/wiki/Precision_and_recall



http://csb.stanford.edu/class/public/lectures/lec4/Lecture6/Data_Visualization/pages/Roc_Curve_Examples.html

ML Process

Model Evaluation

- Performance M**etrics**
  - **Classification problems**
    - **Learning class probabilities**
      - Instead of black and white classification (belongs or doesn't belong to class), learn the **probability of belonging to each class**
      - E.g., Naïve Bayes, SVMs can output class probabilities
      - **Metrics** [Witten et al., 2011, pp. 159-163]
        - » **Quadratic Loss Function**
        - » **Informational Loss Function**

ML Process

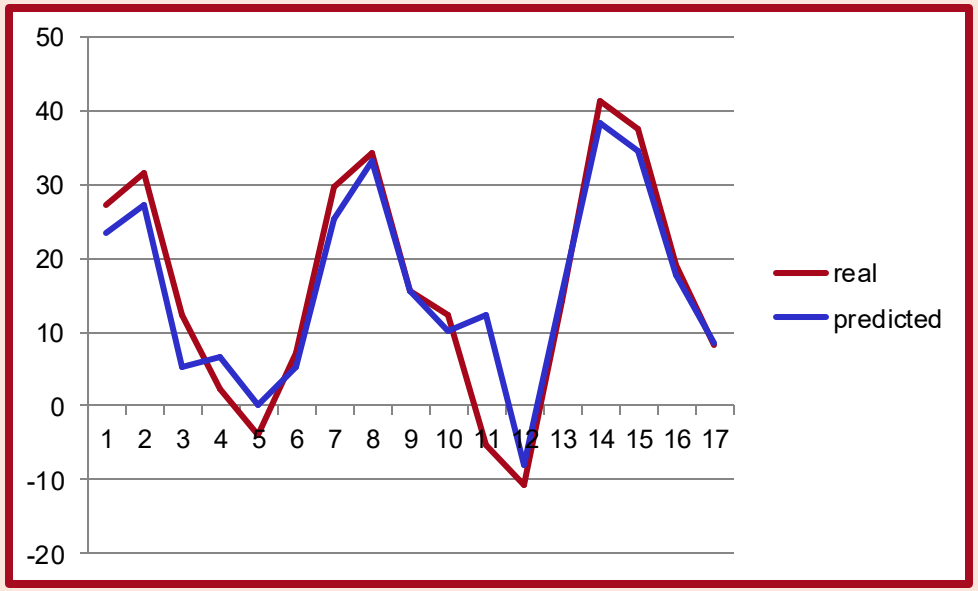
Model Evaluation

- Performance M**etrics**
  - **Regression problems**

Sample nr.	Real Temp	Predicted Temp
1	27.2	23.4
2	31.4	27.2
3	12.3	15.4
4	2.4	0.1
5	-3.8	0.2
6	7.2	5.3
7	29.7	25.4
8	34.2	33.2
9	15.6	15.6
10	12.3	10.1
11	-5.2	-7.2
12	-10.8	-8.1
13	14.2	15.3
14	41.2	38.4
15	37.6	34.5
16	19.2	17.8
17	8.3	8.5

Example:

Predict temperature for next day at 12:00pm:



ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **RMSE**
      - Root (R) mean (M) squared (S) error (E): **metric of average sample error**

$$RMSE = \sqrt{\frac{1}{N} \cdot \sum_{i=1}^{N} (y_i - yp_i)^2}$$

$$RMSE = \sqrt{MSE}$$

$$MSE = \frac{1}{N} \cdot SSE$$

$$SSE = \sum_{i=1}^{N} (y_i - yp_i)^2$$

» y_i: actual value of sample *i*

» yp_i: value predicted for sample *i*

- Goal: **minimize RMSE**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **RMSE**
      - Temperature example
        - » SSE = 479.54
        - » MSE = 28.21
        - » RMSE = 5.31 degrees

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **Pearson's Correlation Coefficient (R)**
      - Measure of the linear correlation between two variables

$$R(y, yp) = \frac{\sum_{i=1}^{N}(y_i - \bar{y}) \cdot (yp_i - \overline{yp})}{\sum_{i=1}^{N}(y_i - \bar{y})^2 \cdot \sum_{i=1}^{N}(yp_i - \overline{yp})^2}$$

        » $\bar{y}$: mean of actual values
        » $\overline{yp}$: mean of actual values
      - Range: [-1, 1]
        » 1: perfect correlation, -1: perfect inverse correlation, 0: no correlation
      - Goal: **maximize R**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **Pearson's Correlation Coefficient (R)**
      - Temperature example
        - » R = 94.4%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **Coefficient of Determination (R^2)**
      - Several different definitions
        - » E.g., square of correlation coefficient
        - » Most common

$$R^2 = 1 - \frac{SSE}{SST}$$

$$SST = \sum_{i=1}^{N} (y_i - \bar{y}_i)^2$$

        - » SST: total sum of squares: (proportional to the sample variance)
        - » Range:]-inf, 1]

Goal: **maximize R^2**

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **Coefficient of Determination (R^2)**
      - Temperature example
        - » $\bar{y}$: 16.06
        - » SSE = 479.54
        - » SST = 3905.5
        - » R^2 = 87.72%

ML Process

Model Evaluation

- Performance M**etrics**
  - **Regression problems**
    - **Other metrics** [Witten et al., 2011, pp. 166-178]
      - Mean Absolute Error
      - Relative Absolute Error
      - Relative Squared Error
      - Root Relative Squared Error

ML Process

Model Evaluation

- Performance M**etrics**
  - **Clustering problems**
    - Idea: **strong intra-cluster cohesion** (i.,e., dense clusters) and **weak inter-cluster coupling** (i.e., well-spaced clusters) → good clustering

ML Process

Model Evaluation

- Performance M**etrics**
  - **Clustering problems**
    - **Silhouette Coefficient (SC)**

$$SC = \max_k \tilde{S}(k)$$

  - $\tilde{S}(k)$: Mean silhouette, *S*(*i*), of all points in cluster *k*
  - **Silhouette** of point *i, S*(*i*)

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad S(i) \in [-1, 1]$$

    - » Measures how well assigned each point *i* is
      - » *a*(*i*): average distance of point *i* to all other points in its cluster
      - » *b*(*i*): smallest average distance of *i* to all points in any other cluster

ML Process

Model Evaluation

- Performance M**etrics**
  - **Clustering problems**
    - **Silhouette Coefficient (SC)**
      - Pros
        - » **Good and intuitive metric** of clustering quality
      - Cons
        - » **Computational cost**: it requires computing the distance between each point and all the others

- Performance M**etrics**
  - **Clustering problems**
    - **Davies-Bouldin Index**

$$DB = \frac{1}{C}\sum_{i=1}^{C} \max_{j \neq i}\left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)}\right)$$

- C: number of clusters
- σ_i: average distance of all points in cluster i from the cluster centroid c_i

      - Low DB → good clustering
    - **Other metrics**
      - Dunn Index (same idea of strong intra-cluster cohesion and weak inter-cluster coupling)

ML Process

Model Evaluation

- Comparison of Different Models
  - **Statistical tests**
    - Guarantee that the **observed differences are not caused by chance effects**
    - **Methods** [Witten et al., 2011, pp. 166-178]
      - **Student's T-test**
      - **Paired T-test**

ML Process

Model Evaluation

- Are results acceptable?
  - **What are acceptable results?**
    - **Perfect results**
      - 100% accuracy ☺
    - Results that **outperform the state-of-the-art**
      - **Accuracy, generalization capability, decision speed**, etc.
      - E.g., human credit decisions, diagnosis, etc.

ML Process

Model Evaluation

- Are results acceptable?
  - Yes → Deploy system

ML Process *Model Evaluation*

- Are results acceptable?
  - No → **Go back and repeat the necessary steps**: find out causes and attempt to fix…
    - **Missing features?**
      - Critical features to the concept may be missing, e.g., articulation is difficult to extract from audio but is important to model emotion → devise methods to obtain the missing features
    - **Redundant features still present?**
      - Repeat feature selection with different algorithms, use domain knowledge about important features, etc.

ML Process *Model Evaluation*

- Are results acceptable?
  - No → **Go back and repeat the necessary steps**: find out causes and attempt to fix…
    - **Error in measurements too high?**
      - E.g., tempo estimation in audio can be error-prone → improve measurement methods, manually correct measurements, …
    - **Bad annotations?**
      - Output values badly assigned (subjective concept, annotations by non-experts, etc.) → repeat annotation experiment

ML Process *Model Evaluation*

- Are results acceptable?
  - No → **Go back and repeat the necessary steps**: find out causes and attempt to fix…
    - **Data acquisition poorly conducted?**
      - Samples might not be representative of the concept, many outliers, narrow range of samples, … → repeat data acquisition
    - **Inadequate model?**
      - E.g., linear model to fit to non-linear data → experiment with different models
    - …

ML Process *Model Deployment*

- Goals
  - Put the **learned model** into real-world **production**
  - **Support** actual **decision-making**

Machine learning tools should be used for **decision-support**, not **decision-making**. **Human experts** must have the **final word**, especially in critical cases (e.g., health)!

ML Process *Model Deployment*

- How?
  - Simple written set of rules for decision-making
  - **Complex piece of software**
    - Automatic classification, prediction, clustering, association rules, etc. for **new, real-world data**
  - **Validation** by **human expert** typically necessary
    - **Models are imperfect** → **human validation** often necessary, especially for **critical tasks**
      - E.g., medical diagnosis
    - → Usually, models are **not completely autonomous**, instead, **support decision-making** by a human

ML Process *Model Deployment*

- Final Model
  - Use the **entire dataset** to learn the **final model**
    - With the selected features
    - With the optimal parameters determined
    - **Expected performance**
      - The average cross-validation performance